

## ***Modelo desde nivel piso***

# Guía Técnica de Funcionamiento: WebXR Inline (Modo Táctil)

A Este documento describe la arquitectura y el flujo lógico del archivo index.html diseñado para renderizar un entorno 3D monoscópico (no estéreo) utilizando la API nativa de WebXR en modo plano (Magic Window), con control de cámara asistido mediante eventos táctiles.

# 1. Configuraciones del Encabezado (<head>)

- **viewport-fit=cover**: Fuerza al lienzo (<canvas>) a expandirse por debajo de las áreas físicas del teléfono (como el "notch" o la cámara perforada en las pantallas de los dispositivos teléfono).
- **mobile-web-app-capable** y **apple-mobile-web-app-capable**: Permiten que la página se comporte como una aplicación nativa (PWA) si el usuario la añade a la pantalla de inicio de su teléfono, ocultando las barras del navegador.

## 2. Módulos e Importaciones (import)

El script se ejecuta como un módulo de JavaScript (`type="module"`), lo que permite traer dependencias externas empaquetadas:

- WebXRButton: Script auxiliar que genera e inyecta el botón gráfico en la interfaz. Gestiona los estados del botón ("INICIAR PANORAMA", "XR NO SOPORTADO", "SALIR") y asegura que la sesión se abra bajo un gesto de usuario obligatorio (el clic en pantalla). WebXRButton: Script auxiliar que genera e inyecta el botón gráfico en la interfaz. Gestiona los estados del botón ("INICIAR PANORAMA", "XR NO SOPORTADO", "SALIR") y asegura que la sesión se abra bajo un gesto de usuario obligatorio (el clic en pantalla).
- Scene, Renderer, createWebGLContext: Componentes base del motor gráfico del repositorio. Configuran el búfer de dibujo, la iluminación, el procesamiento de materiales y la renderización en la tarjeta gráfica.
- Gltf2Node: Cargador especializado para interpretar archivos en formato .gltf. Convierte tu modelo geométrico del campamento (camp.gltf) en vértices legibles por WebGL.

## 2. Módulos e Importaciones (import) ..Continúa

- SkyboxNode: Nodo diseñado para proyectar la textura esférica de 360 grados (eilenriede-park-2k.png) envolviendo toda la escena en el infinito a modo de cielo.
- InlineViewerHelper: El componente clave de esta versión. Es una biblioteca matemática que intercepta la cámara del lienzo. Desactiva el giroscopio físico y mapea las coordenadas del dedo (en móviles) o del ratón (en PC) para rotar la matriz de visualización en 360°.
- WebXRPolyfill: Un emulador en JavaScript que inyecta la API de WebXR en navegadores antiguos o terminales que no la soportan de manera nativa de fábrica.

## 3. Procesamiento de la Textura y el Entorno (Skybox)

Dentro de la fase de carga del escenario (líneas 51 y 52 del código original), la textura de fondo se gestiona a través del nodo SkyboxNode:

- Especificación de la imagen: Carga el archivo físico `media/textures/eilenriede-park-2k.png`. Esta es una imagen de proyección equirectangular (un panorama plano de 360 grados estirado en formato 2:1).
- Mapeo en la GPU: El componente SkyboxNode de tu biblioteca toma esa imagen plana y, mediante un sombreador (shader) interno de WebGL, la "proyecta" en el interior de una esfera virtual gigante que envuelve por completo la escena 3D.
- Efecto de Infinito: El motor gráfico bloquea la posición de esta esfera con respecto a la cámara. Esto significa que aunque camines o deslices el dedo para moverte por el campamento, el fondo (el cielo y el parque lejano) siempre se mantendrá a la misma distancia infinita, simulando un horizonte real de 360 grados.

## 4. Flujo Lógico de Ejecución (Ciclo de Vida)

Paso A: Inicialización (initXR())

- Es el punto de partida del programa (línea final). Ejecuta dos acciones:
  1. Instancia el botón físico WebXRButton y lo añade dentro de la etiqueta <header>.
  2. Llama a navigator.xr.isSessionSupported('inline') para preguntarle al navegador si el teléfono es capaz de procesar gráficos 3D directamente sobre la pantalla plana. Si es compatible, enciende el botón.

## 4. Flujo Lógico de Ejecución (Ciclo de Vida)

Paso B: Solicitud de Sesión (onRequestSession())

- Cuando presionas el botón "INICIAR PANORAMA", se activa esta función.
  1. Ejecuta `navigator.xr.requestSession('inline', { requiredFeatures: ['viewer'] })`.
  2. ¿Por qué 'inline'? Porque le dice al navegador que no divida la pantalla en dos lentes para casco, sino que mantenga un solo canal gráfico visible para el ojo humano de forma monoscópica.
  3. Se añade la característica obligatoria ['viewer'] para desbloquear los permisos internos del navegador para la cámara base.



## 4. Flujo Lógico de Ejecución (Ciclo de Vida)

Paso C: Configuración del Entorno (onSessionStarted())

- Una vez concedida la sesión por el navegador de tu teléfono:
  1. Se asigna la sesión activa a la variable `xrSession` y se añade un oyente (`addEventListener('end')`) para limpiar el sistema si el usuario sale.
  2. Se ejecuta `initGL()`, el cual crea el contexto gráfico compatible con XR y mete el `<canvas>` al cuerpo de la página web de forma visible.
  3. Se crea el `XRWebGLLayer`. Este objeto actúa como el "puente de renderizado", diciéndole a la tarjeta gráfica del móvil que todo lo que dibuje el motor debe enviarse directo a la pantalla de la sesión de WebXR.
  4. Se solicita el espacio de referencia 'viewer'. Al ser concedido, se inicializa el objeto `inlineViewerHelper` pasándole el lienzo y una altura fija de 1.6 metros (simulando la altura promedio de los ojos de una persona de pie en el campamento).

## 4. Flujo Lógico de Ejecución (Ciclo de Vida)

Paso D: El Bucle de Renderizado (`onXRFrame()`)

- Esta función se ejecuta de forma cíclica aproximadamente 60 o 90 veces por segundo (dependiendo de la pantalla de tu Teléfono):
  1. Solicita inmediatamente el siguiente cuadro de animación mediante `session.requestAnimationFrame(onXRFrame)` para mantener el ciclo infinito activo.
  2. Consulta el objeto `inlineViewerHelper.referenceSpace` para verificar si el usuario ha arrastrado el dedo.
  3. Ejecuta `frame.getViewerPose(refSpace)`. Si el usuario movió el dedo, esta función devuelve una matriz de posición modificada táctilmente (pose).
  4. Finalmente, `scene.drawXRFrame(frame, pose)` toma la matriz táctil, calcula la perspectiva de los árboles y las rocas, y dibuja la escena final que ves en pantalla.

## 4. Código QR



# Referencias Bibliográficas